

Straep — API Design

Version: 1.0

Document owner: Engineering Team

Base URL: `https://api.straep.com/v1`

Protocol: HTTPS only

Format: JSON request and response bodies

Authentication: JWT Bearer token (`Authorization: Bearer <access_token>`)

Versioning: URL path versioning (`/v1/`, `/v2/`) for future breaking changes

Table of Contents

- 1. [API Conventions](#)
 - 2. [Authentication Endpoints](#)
 - 3. [User and Profile Endpoints](#)
 - 4. [Brand Page Endpoints](#)
 - 5. [Storefront — Products and Services](#)
 - 6. [Inquiry and Lead Endpoints](#)
 - 7. [Feed and Post Endpoints](#)
 - 8. [Discover and Search Endpoints](#)
 - 9. [Social Graph — Follows, Likes, Saves](#)
 - 10. [Subscription and Payment Endpoints](#)
 - 11. [Template Marketplace Endpoints](#)
 - 12. [Notification Endpoints](#)
 - 13. [Admin Endpoints](#)
 - 14. [Error Response Format](#)
 - 15. [Rate Limiting](#)
 - 16. [Pagination](#)
 - 17. [Webhook Events](#)
-

1. API Conventions

1.1 Request Headers

Header	Required	Value
<code>Content-Type</code>	Yes (POST/PUT/PATCH)	<code>application/json</code>
<code>Authorization</code>	On protected routes	<code>Bearer <access_token></code>
<code>X-Straep-Version</code>	Optional	<code>2025-02-01</code> (date-based API version pin)
<code>Accept-Language</code>	Optional	<code>en-NG</code> , <code>en-US</code> , etc.

1.2 Response Envelope

All successful responses follow this structure:

```
json
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 145,
    "next_cursor": "eyJpZCI6IjEyMyJ9"
  }
}
```

The `meta` key is only present on paginated responses. `data` is either an object or an array depending on the endpoint.

1.3 Date and Time

All timestamps use ISO 8601 format with UTC timezone: `2025-02-10T14:30:00Z`.

1.4 IDs

All entity IDs are UUIDs: `"id": "550e8400-e29b-41d4-a716-446655440000"`.

1.5 HTTP Methods

Method	Use
GET	Retrieve data. Never modifies state.
POST	Create a new resource.
PUT	Replace an entire resource.
PATCH	Partial update of a resource.
DELETE	Delete a resource (soft delete where applicable).

1.6 Empty Fields

Fields that are `null` or empty are included in responses (not omitted). This prevents client-side null-check errors.

2. Authentication Endpoints

POST `/auth/register`

Register a new user account.

Auth required: No

Request body:

json

```
{
  "email": "user@example.com",
  "password": "SecurePass123",
  "account_type": "business"
}
```

Or for phone:

json

```
{
  "phone": "+2348012345678",
  "password": "SecurePass123",
  "account_type": "personal"
}
```

Response **201 Created**:

json

```
{
  "success": true,
  "data": {
    "user_id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "user@example.com",
    "account_type": "business",
    "is_verified": false,
    "onboarding_step": "verify_email",
    "message": "Verification code sent to user@example.com"
  }
}
```

Errors:

- **400** — Validation error (weak password, missing fields).
- **409** — Email or phone already registered.

POST **/auth/verify**

Verify email or phone with OTP.

Auth required: No

Request body:

json

```
{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "code": "482931",
  "channel": "email"
}
```

Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "user_id": "550e8400-...",
    "is_verified": true,
    "onboarding_step": "account_type",
    "access_token": "eyJhbGciOi...",
    "refresh_token": "eyJhbGciOi..."
  }
}
```

Notes: The refresh token is also set as an HTTP-only cookie (`straep_refresh`) in addition to being returned in the body.

POST `/auth/verify/resend`

Resend OTP. Rate limited to once per 60 seconds per user.

Request body: `{ "user_id": "...", "channel": "email" }`

Response (200 OK): `{ "success": true, "data": { "message": "Code resent." } }`

POST `/auth/login`

Login with email/password or phone/password.

Auth required: No

Request body:

```
json
{
  "email": "user@example.com",
  "password": "SecurePass123",
  "remember_me": true
}
```

Response (200 OK):

```
json
```

```

{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOi...",
    "refresh_token": "eyJhbGciOi...",
    "user": {
      "id": "550e8400-...",
      "email": "user@example.com",
      "account_type": "business",
      "onboarding_done": true,
      "plan": "professional",
      "plan_features": {
        "full_customisation": true,
        "premium_templates": true,
        "max_products": 20,
        "advanced_analytics": true
      }
    }
  }
}

```

Note: `plan_features` is computed from the user's active subscription and returned with every login. The frontend stores this to gate features without additional API calls.

Errors:

- `401` — Invalid credentials.
- `403` — Account suspended.
- `429` — Too many failed attempts; account locked.

POST `/auth/google`

OAuth login/signup with Google.

Request body: `{ "id_token": "google_id_token_here" }`

Response: Same structure as `/auth/login`.

POST `/auth/refresh`

Refresh an expired access token.

Auth required: No (uses refresh token from cookie or body)

Request body (optional, if not using cookie): `{ "refresh_token": "eyJhbGciOi..." }`

Response `200 OK`:

json

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOi...",
    "expires_in": 900
  }
}
```

Errors:

- `401` — Refresh token invalid or expired. User must log in again.

POST `/auth/logout`

Revoke the current refresh token.

Auth required: Yes

Response `200 OK`: `{ "success": true, "data": { "message": "Logged out." } }`

POST `/auth/forgot-password`

Request body: `{ "email": "user@example.com" }`

Response `200 OK`: Always returns success to prevent email enumeration. `{ "success": true, "data": { "message": "If an account exists, a reset link has been sent." } }`

POST `/auth/reset-password`

Request body: `{ "token": "reset_token_from_email", "password": "NewPass456" }`

Response `200 OK`: `{ "success": true, "data": { "message": "Password updated." } }`

GET `/auth/me`

Get the currently authenticated user's profile and plan details.

Auth required: Yes

Response `200 OK`:

json

```
{
  "success": true,
  "data": {
    "id": "550e8400-...",
    "email": "user@example.com",
    "account_type": "business",
    "onboarding_done": true,
    "profile": {
      "display_name": "Tunde Design Studio",
      "username": "tunde_designs",
      "avatar_url": "https://res.cloudinary.com/straep/...",
      "bio": "We design brands that stand out."
    },
    "subscription": {
      "plan": "professional",
      "status": "active",
      "current_period_end": "2025-03-10T00:00:00Z",
      "billing_cycle": "monthly"
    },
    "plan_features": {
      "full_customisation": true,
      "premium_templates": true,
      "max_products": 20,
      "max_services": 10,
      "advanced_analytics": true,
      "remove_branding": true,
      "lead_capture": true,
      "max_active_leads": null
    }
  }
}
```

3. User and Profile Endpoints

GET `/users/:username`

Get a public user/profile by username.

Auth required: No

Response `200 OK`: Returns public profile fields (no email, no plan details).

PATCH `/users/me/profile`

Update the current user's profile.

Auth required: Yes

Request body (all fields optional):

json

```
{
  "display_name": "Tunde Design Studio",
  "bio": "We design brands that stand out.",
  "location_city": "Lagos",
  "location_country": "Nigeria",
  "avatar_url": "https://res.cloudinary.com/...",
  "interests": ["fashion", "branding", "design"]
}
```

Response (200 OK): Updated profile object.

POST (/users/me/avatar)

Upload profile avatar. Accepts (multipart/form-data).

Auth required: Yes

Request body: (form-data) with (file) field (image, max 5MB).

Response (200 OK): { "avatar_url": "https://res.cloudinary.com/..." }

Note: Upload the file directly to Cloudinary from the client using a signed upload preset. This endpoint generates and returns the signed upload params:

POST (/users/me/avatar/upload-params)

Response:

json

```
{
  "success": true,
  "data": {
    "cloudinary_url": "https://api.cloudinary.com/v1_1/straep/image/upload",
    "upload_preset": "straep_avatars",
    "signature": "abc123...",
    "timestamp": 1707567890,
    "api_key": "123456789"
  }
}
```

The client uploads directly to Cloudinary, then sends the resulting URL to (PATCH /users/me/profile).

PATCH (/users/me/account)

Update account credentials.

Auth required: Yes

Request body:

json

```
{
  "current_password": "OldPass123",
  "new_password": "NewPass456"
}
```

Or to update email:

json

```
{
  "current_password": "OldPass123",
  "new_email": "new@example.com"
}
```

Changing email triggers a new verification OTP.

4. Brand Page Endpoints

GET `/brands/:username`

Get a public brand page by username slug.

Auth required: No

Cache: Redis, 5-minute TTL. Invalidated on publish.

Response `200 OK`:

json

```

{
  "success": true,
  "data": {
    "id": "brand-uuid",
    "username_slug": "tunde_designs",
    "business_name": "Tunde Design Studio",
    "tagline": "Where brands come to life.",
    "description": "We are a Lagos-based design studio...",
    "category": { "id": "cat-uuid", "name": "Creative Services" },
    "location_city": "Lagos",
    "location_country": "Nigeria",
    "logo_url": "https://res.cloudinary.com/...",
    "cover_url": "https://res.cloudinary.com/...",
    "is_verified": true,
    "follower_count": 284,
    "avg_rating": 4.7,
    "review_count": 23,
    "response_rate": 92.5,
    "contact_email": "hello@tunde.design",
    "whatsapp_number": "+2348012345678",
    "instagram_url": "https://instagram.com/tunde_designs",
    "sections": [ ... ],
    "current_user_follows": false,
    "current_user_saved": false
  }
}

```

Note: `current_user_follows` and `current_user_saved` are included only when the request is authenticated.

POST `/brands`

Create a new brand page.

Auth required: Yes

Plan check: User must have a business, creator, or agency account type.

Request body:

```

json

{
  "business_name": "Tunde Design Studio",
  "username_slug": "tunde_designs",
  "category_id": "cat-uuid",
  "industry": "Creative Services",
  "location_city": "Lagos",
  "location_country": "Nigeria",
  "description": "We are a Lagos-based design studio.",
  "contact_email": "hello@tunde.design"
}

```

Response `201 Created`: Full brand page object.

Errors:

- `409` — Username slug already taken.
 - `403` — Account type does not allow brand page creation.
 - `400` — Plan limit reached (e.g., agency at max brand pages for their plan).
-

PUT `/brands/:id`

Full update of a brand page (not sections — use `/brands/:id/sections` for sections).

Auth required: Yes (owner only)

Request body: Any combination of brand page fields (same as POST, all optional).

Response `200 OK`: Updated brand page object.

Plan checks:

- `brand_color_primary`, `brand_color_secondary`, `heading_font`, `body_font`, `remove_branding` → require Professional plan.
 - If Free plan attempts to set these: `403` with `plan_required: "professional"`.
-

PATCH `/brands/:id/sections`

Update the page sections JSON (autosave from the editor).

Auth required: Yes (owner only)

Request body:

```
json
{
  "draft_sections_json": [ ... ]
}
```

Response `200 OK`: `{ "success": true, "data": { "draft_saved_at": "2025-02-10T14:30:00Z" } }`

POST `/brands/:id/publish`

Publish the current draft sections as the live page.

Auth required: Yes (owner only)

Response `200 OK`:

```
json
```

```
{
  "success": true,
  "data": {
    "is_published": true,
    "published_at": "2025-02-10T14:32:00Z",
    "public_url": "https://straep.com/p/tunde_designs"
  }
}
```

Side effects: Invalidates the Redis cache for this brand page.

DELETE `/brands/:id`

Soft-delete a brand page.

Auth required: Yes (owner only)

Response `(200 OK)`: `{ "success": true, "data": { "message": "Brand page deleted." } }`

Side effects: Page is hidden from public; followers are retained; `deleted_at` is set.

POST `/brands/upload-params`

Get signed Cloudinary upload parameters for logo or cover image.

Auth required: Yes

Request body: `{ "upload_type": "logo" }` or `{ "upload_type": "cover" }`

Response: Signed Cloudinary upload params (same structure as avatar upload params).

GET `/brands/:id/analytics`

Get analytics for the brand page (owner only).

Auth required: Yes (owner only)

Plan check: Basic on Free; full date-range on Professional+.

Query params:

- `period`: `7d`, `30d`, `90d` (default: `7d`; `30d` and `90d` require Professional+)
- `metric`: `views`, `followers`, `inquiries`, `engagement` (default: all)

Response `(200 OK)`:

json

```
{
  "success": true,
  "data": {
    "period": "7d",
    "summary": {
      "profile_views": 1240,
      "profile_views_change": 12.5,
      "new_followers": 23,
      "new_followers_change": 4.2,
      "post_engagement_rate": 4.2,
      "inquiry_count": 8,
      "inquiry_count_change": -2.1,
      "conversion_rate": 35.0
    },
    "daily": [
      { "date": "2025-02-04", "views": 180, "followers": 3, "inquiries": 1 },
      { "date": "2025-02-05", "views": 210, "followers": 5, "inquiries": 2 }
    ],
    "top_products": [
      { "product_id": "...", "name": "Logo Design Package", "views": 84, "inquiries": 1 }
    ],
    "top_posts": [
      { "post_id": "...", "type": "image", "likes": 42, "saves": 8, "reach": 38 }
    ]
  }
}
```

5. Storefront — Products and Services

GET `/brands/:brandId/products`

List all products for a brand (public).

Auth required: No

Query params: `?page=1&limit=12&featured=true`

Response `200 OK`: Paginated list of product objects.

POST `/brands/:brandId/products`

Create a product.

Auth required: Yes (brand owner only)

Plan check: Enforces `max_products` limit.

Request body:

```
json
```

```
{
  "name": "Logo Design Package",
  "description": "A full brand identity package including primary logo, variati
  "price": 85000.00,
  "currency": "NGN",
  "category": "Branding",
  "images": ["https://res.cloudinary.com/..."],
  "availability": "in_stock",
  "delivery_note": "Delivered within 5-7 business days.",
  "is_featured": false
}
```

Response **(201 Created)**: Full product object.

Errors:

- **(403)** — Product limit reached. Returns **{ "plan_required": "professional", "current_limit": 5, "plan_limit": 5 }**.

GET **(/brands/:brandId/products/:productId)**

Get a single product (public).

Auth required: No

Side effect: Increments **(product.view_count)** asynchronously via Celery.

PUT **(/brands/:brandId/products/:productId)**

Full update of a product.

Auth required: Yes (brand owner only)

DELETE **(/brands/:brandId/products/:productId)**

Soft-delete a product (**(deleted_at)** set). Does not decrement the product count towards the plan limit.

Auth required: Yes (brand owner only)

POST **(/brands/:brandId/products/:productId/archive)**

Toggle product archive status (hidden from storefront but not deleted).

Auth required: Yes (brand owner only)

GET `/brands/:brandId/services`

List all services for a brand (public).

POST `/brands/:brandId/services`

Create a service. Plan check enforces `max_services`.

Request body:

json

```
{
  "name": "Brand Strategy Consultation",
  "description": "A 90-minute session covering brand positioning, messaging, and",
  "starting_price": 45000.00,
  "currency": "NGN",
  "duration_label": "90 minutes",
  "consultation_available": true,
  "cta_type": "inquiry"
}
```

PUT `/brands/:brandId/services/:serviceId`

Update a service.

DELETE `/brands/:brandId/services/:serviceId`

Soft-delete a service.

6. Inquiry and Lead Endpoints

POST `/brands/:brandId/inquiries`

Submit an inquiry to a brand.

Auth required: No (guests allowed)

Rate limit: 3 inquiries per IP per brand per 24 hours.

Request body:

json

```
{
  "sender_name": "Chisom Okafor",
  "sender_email": "chisom@example.com",
  "message": "Hi, I'm interested in your logo design package. Could you share y
  "product_interest": "Logo Design Package",
  "product_id": "product-uuid-optional",
  "contact_preference": "whatsapp",
  "captcha_token": "cf_turnstile_token"
}
```

Response **201 Created**:

```
json
{
  "success": true,
  "data": {
    "inquiry_id": "inq-uuid",
    "message": "Inquiry sent successfully.",
    "brand_avg_response_hours": 4.2
  }
}
```

Side effects:

- Creates **Inquiry** record.
- Queues Celery task: send email notification to brand owner.
- Creates in-app notification for brand owner.
- Increments **brand_page.inquiry_count**.
- Increments **product.inquiry_count** if **product_id** is set.

GET **/dashboard/leads**

Get all leads/inquiries for the authenticated brand.

Auth required: Yes (business/creator/agency)

Plan check: Free plan returns max 10 leads; includes **hidden_count** if more exist.

Query params:

- **status**: **new**, **contacted**, **in_discussion**, **converted**, **closed**, **rejected**
- **page**, **limit**
- **search**: searches sender name and message content
- **date_from**, **date_to**

Response **200 OK**:

```
json
```



```
{
  "success": true,
  "data": {
    "inquiries": [
      {
        "id": "inq-uuid",
        "sender_name": "Chisom Okafor",
        "sender_user": null,
        "product_interest": "Logo Design Package",
        "status": "new",
        "is_read": false,
        "contact_preference": "whatsapp",
        "message_preview": "Hi, I'm interested in your logo design package...",
        "created_at": "2025-02-10T14:30:00Z"
      }
    ],
    "hidden_count": 4,
    "total_new": 3
  },
  "meta": { "page": 1, "limit": 20, "total": 18 }
}
```

GET `/dashboard/leads/:inquiryId`

Get full lead detail.

Auth required: Yes (brand owner of this inquiry only)

Response `(200 OK)`: Full inquiry object including notes.

Side effect: Sets `(inquiry.is_read = true)`.

PATCH `/dashboard/leads/:inquiryId`

Update lead status or notes.

Auth required: Yes (brand owner only)

Request body:

```
json
{
  "status": "converted",
  "note": "Client signed the contract on Feb 12. Logo delivery by Feb 19."
}
```

Response `(200 OK)`: Updated inquiry object.

Side effects: Creates `(LeadStatusHistory)` entry. If status is `(converted)`, updates brand's conversion rate metric.

7. Feed and Post Endpoints

GET `/feed`

Get the personalised home feed for the authenticated user.

Auth required: Yes

Pagination: Cursor-based. Pass `?cursor=<next_cursor>` for the next page.

Response `200 OK`:

```
json
{
  "success": true,
  "data": {
    "posts": [
      {
        "id": "post-uuid",
        "author": {
          "id": "user-uuid",
          "display_name": "Tunde Design Studio",
          "username": "tunde_designs",
          "avatar_url": "...",
          "is_verified": true,
          "current_user_follows": true
        },
        "post_type": "image",
        "content": "New project reveal! Check out this brand identity we design",
        "media_urls": ["https://res.cloudinary.com/..."],
        "like_count": 42,
        "comment_count": 8,
        "save_count": 5,
        "current_user_liked": true,
        "current_user_saved": false,
        "created_at": "2025-02-10T10:00:00Z",
        "product": null
      }
    ]
  },
  "meta": {
    "next_cursor": "eyJpZCI6InBvc3QtdXVpZC0yIn0=",
    "has_more": true
  }
}
```

POST `/posts`

Create a new post.

Auth required: Yes (business, creator, agency only for brand posts; personal users in Phase 3)

Request body (image post example):

```
json
{
  "post_type": "image",
  "content": "New project reveal! Check out this brand identity...",
  "media_urls": ["https://res.cloudinary.com/...", "https://res.cloudinary.com/..."],
  "product_id": null
}
```

Request body (offer post example):

```
json
{
  "post_type": "offer",
  "content": "Valentine's sale - 20% off all logo packages!",
  "offer_headline": "Valentine's Design Special",
  "offer_discount": "20% off",
  "offer_code": "VAL20",
  "offer_expires_at": "2025-02-14T23:59:59Z"
}
```

Response `201 Created`: Full post object.

Side effects:

- Sets `post.is_published = true`.
- Queues Celery fan-out task to push to followers' feed caches.
- Increments `brand_page.post_count`.

GET `/posts/:postId`

Get a single post (public if brand page is public).

DELETE `/posts/:postId`

Soft-delete a post.

Auth required: Yes (post author only)

Side effects: Removes from followers' feed caches.

GET `/posts/:postId/comments`

Get comments for a post (paginated, cursor-based).

Auth required: No

POST `/posts/:postId/comments`

Add a comment.

Auth required: Yes

Request body: `{ "content": "Love this project!", "parent_id": null }`

Response `201 Created`: Comment object.

DELETE `/posts/:postId/comments/:commentId`

Delete own comment.

POST `/posts/upload-params`

Get signed Cloudinary params for media upload.

Request body: `{ "file_type": "image", "count": 3 }`

Response: Array of signed upload param sets (one per file).

8. Discover and Search Endpoints

GET `/discover`

Get the discover page content.

Auth required: No

Response `200 OK`:

```
json
{
  "success": true,
  "data": {
    "featured_brands": [ ... ],
    "trending_brands": [ ... ],
    "verified_brands": [ ... ],
    "new_brands": [ ... ],
    "recommended_for_you": [ ... ]
  }
}
```

`recommended_for_you` is only populated when the user is authenticated.

GET `/search`

Search brands, products, and services.

Auth required: No

Query params:

- `q` (required): Search query string.
- `type`: `brands`, `products`, `services` (default: all, returned in separate arrays).
- `category`: category slug.
- `location`: city name.
- `verified`: `true`/`false`.
- `has_offers`: `true`/`false`.
- `min_rating`: `1`-`5` (Phase 3).
- `min_price`, `max_price`: for product/service searches.
- `sort`: `popular`, `newest`, `rating`.
- `page`, `limit`.

Response `200 OK`:

```
json
{
  "success": true,
  "data": {
    "brands": {
      "results": [ ... ],
      "total": 24
    },
    "products": {
      "results": [ ... ],
      "total": 8
    },
    "services": {
      "results": [ ... ],
      "total": 5
    }
  },
  "meta": { "page": 1, "limit": 20 }
}
```

GET `/categories`

List all platform categories.

Auth required: No

Response `200 OK`:

json

```
{
  "success": true,
  "data": [
    { "id": "cat-uuid", "name": "Fashion", "slug": "fashion", "icon_url": "..."},
    { "id": "cat-uuid", "name": "Food & Restaurant", "slug": "food-restaurant",
  ]
}
```

9. Social Graph — Follows, Likes, Saves

POST `/brands/:brandId/follow`

Follow or unfollow a brand (toggles).

Auth required: Yes

Response `200 OK`:

json

```
{
  "success": true,
  "data": {
    "is_following": true,
    "follower_count": 285
  }
}
```

POST `/posts/:postId/like`

Like or unlike a post (toggles).

Auth required: Yes

Response `200 OK`:

json

```
{
  "success": true,
  "data": {
    "is_liked": true,
    "like_count": 43
  }
}
```

Save or unsave a post.

Auth required: Yes

Response (200 OK): { "is_saved": true, "save_count": 6 }

POST (/products/:productId/save)

Save or unsave a product.

Auth required: Yes

POST (/posts/:postId/report)

Report a post.

Auth required: Yes

Request body:

```
json
{
  "reason": "spam",
  "detail": "This post is advertising fake products."
}
```

Response (201 Created): { "success": true, "data": { "message": "Report received. We'll review this post." } }

GET (/users/me/saved)

Get the current user's saved items.

Auth required: Yes

Query params: ?type=products|posts|services

GET (/brands/:brandId/followers)

Get followers of a brand (paginated).

Auth required: No

10. Subscription and Payment Endpoints

GET (/subscriptions/plans)

List all available subscription plans with features and pricing.

Auth required: No

Query params: `?currency=NGN|USD`

Response `200 OK`:

```
json
{
  "success": true,
  "data": [
    {
      "id": "plan-uuid",
      "name": "professional",
      "display_name": "Professional",
      "monthly_price_ngn": 4500.00,
      "yearly_price_ngn": 43200.00,
      "is_most_popular": true,
      "features": {
        "full_customisation": true,
        "premium_templates": true,
        "max_products": 20,
        "max_services": 10,
        "advanced_analytics": true,
        "remove_branding": true
      },
      "feature_list": [
        "Full brand page customisation",
        "Custom colours and fonts",
        "Up to 20 products",
        "Advanced analytics",
        "Remove Straep branding",
        "Unlimited lead management"
      ]
    }
  ]
}
```

GET `/subscriptions/me`

Get the current user's active subscription.

Auth required: Yes

Response `200 OK`: Current subscription object or `null` if on Free plan.

POST `/subscriptions/checkout`

Initiate payment checkout.

Auth required: Yes

Request body:

```
json

{
  "plan_name": "professional",
  "billing_cycle": "monthly",
  "return_to": "/dashboard/brand-page/editor"
}
```

Response **200 OK**:

```
json

{
  "success": true,
  "data": {
    "payment_url": "https://checkout.paystack.com/...",
    "reference": "STR-20250210-XYZ123"
  }
}
```

Note: The frontend redirects the user to `payment_url`. After payment, Paystack redirects to `https://straep.com/dashboard/subscription/callback?reference=STR-20250210-XYZ123`. The callback page shows "Verifying payment..." while the webhook activates the subscription in the background.

POST `/subscriptions/webhook`

Paystack webhook receiver.

Auth required: No (Paystack signature verification instead)

Headers: `X-Paystack-Signature: sha512_hash`

Request body: Raw Paystack webhook payload.

Processing logic:

1. Verify HMAC-SHA512 signature using Paystack secret key.
2. If invalid signature: return 401 immediately.
3. Parse event type:
 - `'charge.success'` → activate/renew subscription.
 - `'subscription.disable'` → set subscription status to cancelled.
 - `'invoice.payment_failed'` → set status to past_due; send payment failed email.
4. Find Payment record by provider_reference. If duplicate webhook: return 200 immediately (idempotent).
5. Update Subscription record.
6. Emit `'subscription_activated'` internal event.
7. Return 200 immediately (Paystack retries if non-200).

POST `/subscriptions/cancel`

Cancel the current subscription.

Auth required: Yes

Request body:

```
json
{
  "reason": "too_expensive",
  "detail": "I'm a small business and can't afford it right now."
}
```

Response `200 OK`:

```
json
{
  "success": true,
  "data": {
    "status": "cancel_at_period_end",
    "active_until": "2025-03-10T00:00:00Z",
    "message": "Your Professional plan will remain active until March 10, 2025."
  }
}
```



POST `/subscriptions/reactivate`

Reactivate a cancelled (but still active) subscription.

Auth required: Yes

Response `200 OK`: Updated subscription object with `cancel_at_period_end: false`.

11. Template Marketplace Endpoints

GET `/templates`

List templates in the marketplace (approved only).

Auth required: No

Query params:

- `category`: template category slug.
- `is_free`: `true`/`false`.

- `sort`: `popular`, `newest`, `rating`.
 - `q`: search by template name.
 - `page`, `limit`.
-

GET `/templates/:templateId`

Get full template details (public).

Auth required: No

Response: Full template object with developer info, preview images, features, and pricing.

POST `/templates/:templateId/install`

Install a free template. Creates a `TemplatePurchase` record with `amount_paid: 0`.

Auth required: Yes

Plan check: User must be on a paid plan to install paid templates.

POST `/templates/:templateId/checkout`

Initiate checkout to purchase a paid template.

Auth required: Yes

Response: Payment URL (same pattern as subscription checkout).

GET `/users/me/templates`

Get the current user's installed/purchased templates.

Auth required: Yes

POST `/users/me/templates/:templateId/apply`

Apply an installed template to the user's brand page.

Auth required: Yes

Request body: `{ "brand_page_id": "brand-uuid" }`

Response `200 OK`: Updated brand page object.

POST `/dev/templates`

Upload a new template (developer only).

Auth required: Yes (developer account only)

Request body:

json

```
{
  "name": "Fashion Store Pro",
  "category": "fashion",
  "short_description": "A bold, image-forward template for fashion brands.",
  "full_description": "## Fashion Store Pro\n\nA modern template...",
  "price_ngn": 2500.00,
  "preview_images": ["https://res.cloudinary.com/..."],
  "package_url": "https://res.cloudinary.com/straep/templates/fashion-store-pro",
  "supported_sections": ["header", "about", "products", "testimonials", "contact"]
}
```

GET `/dev/templates`

Get the developer's own templates.

Auth required: Yes (developer only)

GET `/dev/templates/:templateId/sales`

Get sales analytics for a specific template.

Auth required: Yes (template owner only)

GET `/dev/earnings`

Get total earnings and payout balance.

Auth required: Yes (developer only)

POST `/dev/payouts/request`

Request a payout.

Auth required: Yes (developer only)

Request body:

json

```
{
  "amount": 15000.00,
  "bank_name": "Guaranty Trust Bank",
  "account_number": "0123456789",
  "account_name": "Emmanuel Obi"
}
```

12. Notification Endpoints

GET `/notifications`

Get notifications for the current user.

Auth required: Yes

Query params: `?is_read=false&page=1&limit=20`

Response: Paginated list of notification objects.

POST `/notifications/mark-read`

Mark notifications as read.

Auth required: Yes

Request body:

```
json
{
  "notification_ids": ["notif-uuid-1", "notif-uuid-2"],
  "mark_all": false
}
```

Setting `mark_all: true` marks all unread notifications as read.

GET `/notifications/unread-count`

Get the count of unread notifications (for the badge in the navbar).

Auth required: Yes

Response: `{ "count": 3 }`

13. Admin Endpoints

All admin endpoints require the `is_staff` or `is_superuser` flag on the User. Admin endpoints are under `/admin/` and use the same JWT auth.

GET `/admin/overview`

Platform overview metrics.

Auth required: Yes (admin only)

GET `/admin/users`

List users with filters.

Query params: `{`

`account_type=business&plan=professional&status=active&q=search_term&page=1`

GET `/admin/users/:userId`

Get full user detail including their brand pages, subscriptions, and reports.

POST `/admin/users/:userId/suspend`

Suspend a user.

Request body: `{ "reason": "Repeated spam posts.", "duration_days": 7 }`

POST `/admin/users/:userId/unsuspend`

Lift a suspension.

GET `/admin/templates`

List templates pending review.

Query params: `?status=pending`

POST `/admin/templates/:templateId/approve`

Approve a template. Sets `status = "approved"` and notifies developer.

POST `/admin/templates/:templateId/reject`

Reject a template.

Request body: `{ "reason": "Preview images do not match the template design." }`

GET `/admin/reports`

List reported posts.

Query params: `?status=pending`

POST `/admin/posts/:postId/remove`

Remove a post.

Request body: { "reason": "Violates Straep community guidelines – spam." }

GET /admin/verification

List pending verification requests.

POST /admin/verification/:requestId/approve

Approve a verification request. Sets brand_page.is_verified = true and notifies the brand.

POST /admin/verification/:requestId/reject

Reject a verification request.

Request body: { "reason": "The submitted CAC document could not be verified. Please resubmit with a clearer copy." }

GET /admin/audit-log

Get the audit log.

Query params: ?admin_id=uuid&action=approve_template&date_from=2025-02-01&page=1

14. Error Response Format

All errors follow this format:

```
json
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed.",
    "details": [
      { "field": "email", "message": "A valid email address is required." },
      { "field": "password", "message": "Password must be at least 8 characters" }
    ]
  }
}
```

Standard Error Codes

HTTP Status	Code	Description
400	VALIDATION_ERROR	Request body failed validation
401	AUTHENTICATION_REQUIRED	No valid token provided
401	TOKEN_EXPIRED	Access token has expired (client should refresh)
401	INVALID_CREDENTIALS	Wrong email/password
403	FORBIDDEN	Authenticated but not authorised for this action
403	PLAN_REQUIRED	Feature requires a higher plan
403	ACCOUNT_SUSPENDED	Account has been suspended
404	NOT_FOUND	Resource does not exist
409	CONFLICT	Resource already exists (e.g., username taken)
422	UNPROCESSABLE	Business logic error (e.g., product limit reached)
429	RATE_LIMIT_EXCEEDED	Too many requests
500	SERVER_ERROR	Internal server error

Plan Required Error (403)

```
json
{
  "success": false,
  "error": {
    "code": "PLAN_REQUIRED",
    "message": "This feature requires a Professional plan.",
    "details": {
      "feature": "custom_brand_colours",
      "required_plan": "professional",
      "current_plan": "free",
      "upgrade_url": "https://straep.com/dashboard/subscription/upgrade?feature"
    }
  }
}
```


Endpoint Group	Limit	Window
All unauthenticated endpoints	60 requests	Per minute per IP
All authenticated endpoints	300 requests	Per minute per user
<code>POST /auth/login</code>	5 attempts	Per minute per IP (lockout at 5 failures)
<code>POST /auth/verify/resend</code>	1 request	Per 60 seconds per user
<code>POST /brands/:id/inquiries</code>	3 requests	Per 24 hours per IP per brand
<code>POST /posts</code>	20 posts	Per hour per user
<code>GET /feed</code>	60 requests	Per minute per user
Admin endpoints	300 requests	Per minute per admin user

Rate limit headers returned on every response:

```
X-RateLimit-Limit: 300
X-RateLimit-Remaining: 267
X-RateLimit-Reset: 1707567950
```

On limit exceeded: `429 Too Many Requests` with `Retry-After: 60` header.

16. Pagination

Cursor-Based Pagination (Feed)

Used for the feed where ordering can change.

Request: `GET /feed?cursor=eyJpZCI6InBvc3QtdXVpZC0yIn0=&limit=20`

Response meta:

```
json
{
  "meta": {
    "next_cursor": "eyJpZCI6InBvc3QtdXVpZC00MiJ9",
    "has_more": true
  }
}
```

The cursor is a base64-encoded JSON object pointing to the last item returned. The server decodes it and fetches the next page.

Offset-Based Pagination (Everything Else)

Request: `GET /dashboard/leads?page=2&limit=20`

Response meta:

json

```
"meta": {  
  "page": 2,  
  "limit": 20,  
  "total": 145,  
  "total_pages": 8,  
  "has_next": true,  
  "has_prev": true  
}
```

17. Webhook Events

17.1 Inbound Webhooks (Payment Providers → Straep)

Paystack Events Handled

Event	Action
<code>charge.success</code>	Create Payment record, activate/renew Subscription
<code>subscription.create</code>	Log new subscription via provider
<code>subscription.disable</code>	Set <code>subscription.status = cancelled</code>
<code>invoice.create</code>	Log upcoming charge
<code>invoice.payment_failed</code>	Set <code>status = past_due</code> , send payment failed email
<code>transfer.success</code>	Mark developer payout as completed
<code>transfer.failed</code>	Mark developer payout as failed, notify admin

Webhook Security

python

```
# Django view - /subscriptions/webhook
import hmac
import hashlib

def verify_paystack_signature(request):
    paystack_secret = settings.PAYSTACK_SECRET_KEY
    signature = request.headers.get('X-Paystack-Signature', '')
    body = request.body
    expected = hmac.new(
        paystack_secret.encode('utf-8'),
        body,
        hashlib.sha512
    ).hexdigest()
    return hmac.compare_digest(signature, expected)
```

17.2 Outbound Webhooks (Straep → Developer Apps)

Phase 5 feature — not required for MVP. A developer API with outbound webhooks will be designed when the public API is launched.

End of API Design Document

Next document: 09-Tech-Stack-and-Implementation-Guide.md